CHAPTER 1

DESIGN A PAYMENT SYSTEM

1 Introduction to the Payment System 2
2 Design of a Payment System 7

Explore the functionality and design of secure, efficient payment systems in fintech.

1 Introduction to the Payment System

In today's digital age, financial technology (fintech) is reshaping the landscape of e-commerce and digital marketing. Seamlessly connecting merchants and consumers, fintech enables transactions to happen with just a few clicks, making purchases faster, more convenient, and increasingly secure. Innovations like integrated payment platforms, digital wallets, and fintech-driven gateways are not only enhancing the customer experience but also streamlining business operations.
In this lesson, we'll explore how digital payment systems work and their pivotal role in powering modern commerce.

1.1 What is a digital payment system?

A **payment system** is a mechanism or process that enables individuals, businesses, and organizations to transfer money or make transactions for the purpose of making a purchase. It encompasses various methods and technologies, both digital and traditional, that facilitate the exchange of money or goods. Payment systems encompass a wide range of transactions, including credit and debit card transactions, bank transfers, electronic funds transfers (EFTs), mobile payments, digital wallets, and cryptocurrencies. These systems ensure secure and efficient financial transactions between parties, supporting commerce and economic activities on various scales.
In this design problem, we'll focus on payments made through credit and debit cards.
Let's first understand the payment process in detail before moving to any aspect of the payment System Design.

1.2 The payment process

Multiple entities are involved in the payment process. Assume that a customer wants to pay for goods or services using a credit or debit card. First , they have to enter their credit or debit card details to pay online. However, this complex process entails multiple steps and interacting entities, including cardholder verification, payment processing and validation, and merchant registration.
The following figure illustrates the flow of a payment processing system:

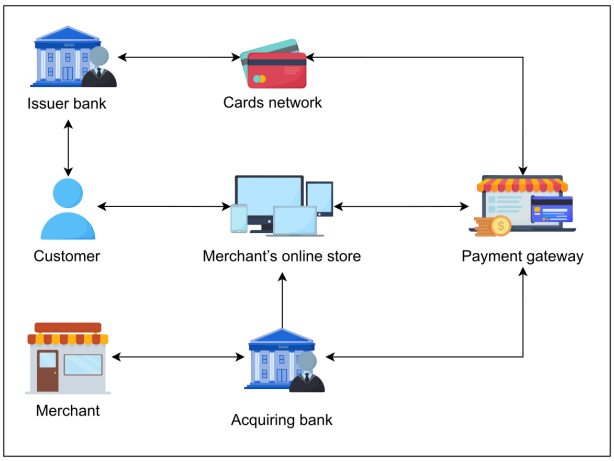

Figure 1.1: Payment processing system and the entities involved in it

The role of different entities involved in the payment processing system is mentioned in the following table:

In a payment system, especially for credit card transactions, the process typically involves two key phases: *authorization* and *settlement*. These phases ensure that a transaction is authorized and then completed by transferring funds from the customer's account to the merchant's account. The following is an overview of each phase:

Entities	Description
Customer	• A cardholder who wants to purchase goods or services.
Merchant	• A company or person who provides goods or services.
Issuer bank	• The bank that holds the customer's account and provides credit or debit cards.
Acquiring bank	• The bank holding the merchant's account.
Merchant's online store	• The online website where the customer purchases goods or services by providing credit card details.
Payment gateway	• A network that facilitates and authorizes transactions of funds to merchants' accounts. • Requests and receives payment from the customer's account.
Cards network	• Validates the credit or debit card information. • Facilitates payments via credit and debit cards. • Sets the terms for credit or debit card transactions.

The authorization phase

The authorization phase is the initial step in a payment transaction. Its primary purpose is to verify whether the customer's payment method (e.g., credit card) is valid, has sufficient funds or credit available, and can be used to make the intended purchase.

1. The customer initiates a payment by providing payment details, such as credit card information, to the merchant's payment system (usually via a payment gateway).
2. The payment gateway sends the payment request to the customer's issuer bank (the bank that issued the credit card).
3. The issuer bank performs several checks, including verifying the card's validity, checking for available credit, and ensuring the transaction is not fraudulent.
4. If all checks pass successfully, the issuing bank sends the authorization code to the payment gateway, indicating that the transaction is approved.
5. The payment gateway relays the authorization code to the merchant, indicating that the customer's payment method has been authorized for the transaction.

If the authorization is successful, the transaction amount is reserved on the customer's credit card but not transferred to the merchant. The funds are held pending until the settlement phase.

The settlement phase

The settlement phase occurs after the authorization phase. It finalizes the payment transaction by transferring the authorized funds from the customer's account to the merchant's.

1. After receiving the authorization code, the merchant accumulates authorized transactions for a certain period (for example, a day's worth).

2. The merchant initiates a batch settlement process, sending all authorized transactions to the payment processor or acquiring bank. In some cases, a payment gateway also functions as a payment processor. For example, Stripe is a payment gateway and a payment processor.

3. The payment processor or acquiring bank verifies the transactions and transfers funds from the customers' issuing banks to the merchant's bank.

4. The acquiring bank or payment processor reconciles the transactions, ensuring all authorized transactions are accounted for and the correct funds are transferred.

5. Once reconciliation is complete, the acquiring bank or payment processor deposits the funds into the merchant's bank account, completing the settlement process.

   Note: The authorization and settlement phases may not occur simultaneously. Authorization is typically immediate and confirms that a valid payment method and funds are available. On the other hand, settlement may occur in batches and is the process of moving the funds to the merchant's account in a bulk transfer.

During the settlement phase, the authorized funds are transferred to the merchant's account, and the payment transaction is considered complete. The customer's credit card statement will reflect the finalized transaction.
In what scenarios could authorization succeed but settlement fail, and how should the system handle it?
Use the AI assessment widget below to submit your solution and get an interactive response.

Question:

The settlement phase

Reference Answer:

Failures can occur due to network issues, bank errors, or insufficient funds at settlement; the system should detect the failure, notify stakeholders, and attempt retries or reversal.

We have explored how the payment processing system works. Now, let's move on to design a payment system by considering the following requirements:

1.3 Requirements

Let's understand the functional and non-functional requirements of the payment system.

Functional requirements

We are designing a payment system to accept credit or debit card payments via a PSP(Payment service provider). Therefore, we will stick to the following functional requirements:

- **User registration and authentication:** Users should be able to create accounts, log in securely, and reset passwords.

- **Payment processing:** Users should be able to initiate payment processing upon purchasing a product or service. The system should accept payment via credit and debit cards.

- **Transaction history:** Users should be able to view a history of their transactions, including dates, amounts, and descriptions.

- **Balance management:** Users should be able to check their account balances, including available funds and pending transactions.

- **Mobile accessibility:** Users should be able to access and use the payment system through mobile apps or responsive web interfaces.

Non-function requirements

The following are the non-functional requirements that the payment system should meet for smooth operations:

- **Data integrity and security:** The system should comply with industry security standards. It should also have robust data encryption and secure storage mechanisms, maintain consistency, and prevent data corruption.
- **Availability:** The system should be highly available to ensure users can always access it.
- **Reliability:** Redundancy, failover mechanisms, and backup systems should be in place.
- **Scalability:** The system should scale seamlessly as the user base and transaction volume grow. It should also handle peak loads during events like promotions or sales.
- **Performance:** The system should be able to handle high transaction loads without experiencing significant slowdowns. The response times for payment processing and account updates should also be within acceptable limits(Payment processing and account updates should occur within a few seconds to ensure a smooth and responsive user experience. Ideally, transactions are completed in under 3 seconds, with account changes reflected instantly or within moments to maintain user trust and system reliability.).

Based on the requirements, we'll estimate the required resources for our system as follows:

1.4 Resource estimation

For resource estimation, we assume that our system can perform 50 million transactions per day, or 2 million transactions per hour. This number could increase to 5 million transactions per hour in peak hours. Assuming these numbers, let's estimate the storage, bandwidth, and required number of servers.

Storage estimation

In the payment system, we need to store various types of data, including users' details and their associated payment information. Also , we would be required to store transaction records. Assume that each transaction takes 100 bytestransaction and each user's record takes up to 200 bytesuserrecord. So , the storage required per day would be as follows:

$$\text{Storage required per day} = 50\text{ M} \times 100\text{ bytes} + 200\text{ bytes} \approx 5\text{ GB}$$

Note: Since each user's record is stored only once and transactions only reference the user ID, the 200 bytes are not repeated across the 50 million transactions.

Below is a calculator to help us estimate our required storage.

Bandwidth estimation

In a payment system, bandwidth ensures timely and reliable communication between clients, the payment gateway, and external systems like banks or PSPs. Each transaction involves data moving into and out of the system, requests sent to process payments, and responses returned with confirmation, status, and audit details. Let 's estimate the incoming and outgoing bandwidths for our system:

Incoming bandwidth As mentioned earlier, our system is designed to support up to 5 million transactions per hour during peak periods. In other words, our system should be capable of handling approximately 1400 transactions per second. Given that the size of each transaction is 100 bytes, which can be calculated as:

$$\begin{aligned}\text{Total}_{\text{incoming bandwidth}} &= \text{Total}_{\text{transactions/sec}} \times \text{Total}_{\text{transaction size}} \\ &= 1400 \times 100 \text{ Bytes} \\ &\approx 1.12 \text{ Mbps}\end{aligned}$$

Outgoing bandwidth The outgoing response typically contains more information, including status, authorization tokens, timestamps, messages, and optional fields such as digital signatures or audit references. On average, we assume 130 bytes(Assumed based on fields such as transaction ID (16 bytes), status code (10 bytes), timestamp (8 bytes), user ID (16 bytes), and metadata or protocol overhead (~80 bytes).) per response.

$$\begin{aligned}\text{Total}_{\text{outgoing bandwidth}} &= \text{Total}_{\text{responses/sec}} \times \text{Total}_{\text{response size}} \\ &= 1400 \times 130 \text{ Bytes} \\ &\approx 1.46 \text{ Mbps}\end{aligned}$$

We can use the following calculator to estimate the incoming and outgoing bandwidths:

Quiz

Question 1: Assume that the number of transactions is increased to 3000 per second in peak time. Also , suppose that each transaction's size is increased to 350 bytes. What would be the bandwidth requirement estimation in this case?

- **[CORRECT]** 8400 $Kbps$
  Explanation: $(3000 \times 350 \times 8)/1000 = 8400\ Kbps$
- 8400 $KBps$
- 8.4 $Kbps$
- None of the above

  Note: Keep in mind that these estimations are fairly basic, and in a real-world situation, we would probably have to consider extra factors like redundancy, backups, indexing, and other metadata.

Number of servers estimation

To estimate the number of servers, we need to know the number of daily active users of the payment system. We assume that our system can perform 50 million transactions per day (50 million daily active users). Considering our assumption of using daily active users as a proxy for the number of requests per second, we get 50 million requests per second. Then , we use the following formula to calculate the number of servers:

$$\text{Servers needed at peak load} = \frac{\text{Number of requests/second}}{\text{RPS of server}}$$

Using 64,000 as an estimated RPS a server can handle, the required servers are estimated as follows:

$$\text{Servers needed at peak load} = \frac{50 \text{ million}}{64{,}000} 782 \text{ servers}$$

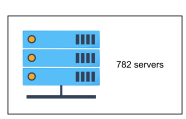

Figure 1.2: Number of servers required for payment system

After discussing the required resources, let's move on to the building blocks we'll use to design the payment system.

1.5 Building blocks we will use

When designing a payment system, we will use the following building blocks. We would also need other essential components and services, including a payment gateway, risk check service, and reconciliation system.

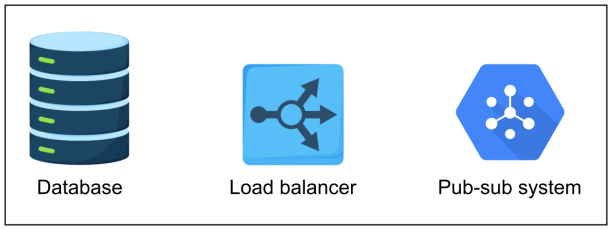

Figure 1.3: Building blocks in the payment System Design

- **Databases** are required to store user-generated payment events and transaction data.
- **Load balancers** must distribute millions of incoming user requests among available servers.
- **Pub-sub systems** decouple essential services such as the payment service, wallet, and ledger services.

In the next lesson, we'll focus on the design of a payment system.

2 Design of a Payment System

In the previous lesson, we explored the working of the payment system, defined key requirements, and estimated different resources. Now , let's expand our discussion by starting with the high-level design of the payment system.

2.1 High-level design

Let's understand the high-level design of the payment system with the following illustration:

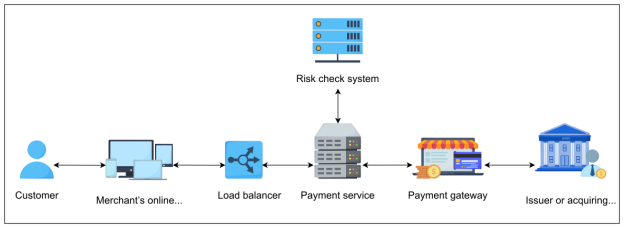

Figure 1.4: High-level design of the payment system

1. When a customer pushes the payment button on the merchant's online store, an event is generated, which triggers the payment service.
2. The payment service processes the customer's necessary information and sends the transaction details to the risk check system for fraud detection and risk assessment.
3. If the transaction passes the risk checks, the payment service forwards the request to the payment gateway.

4. The payment gateway verifies the payment details and checks for correctness. Once validated, the request is forwarded to the issuer's bank.

5. The issuer bank processes the request and sends the required payment to the payment service, which deposits it into the merchant's account.

6. The merchant's account is updated to reflect the credited amount on successful transactions.

How does decoupling fraud checks from the payment gateway improve scalability and security?
Use the AI assessment widget below to submit your solution and get an interactive response.

Question:

Decoupling fraud checks

Reference Answer:

Decoupling fraud checks keeps the gateway lightweight while a separate fraud service performs advanced risk analysis. It improves scalability by allowing each system to scale independently, enhances security by reducing the gateway's attack surface, and increases resilience because fraud failures won't halt payment processing.

We have explored the payment system's high-level design. Before moving to the detailed design, let's discuss some essential APIs, which define users' entry points to the system:

2.2 API design

The following are the essential APIs that meet our functional requirements.

User registration and authentication

The user registration and authentication consist of the following two APIs.

- **User registration:** We have the following API for user registration.

```
registerUser(username, email, password)
```

Under the hood, the `registerUser` API protects the password by hashing it before saving it to the database. The parameters in the API call are explained in the following table:

Parameter	Description
`username`	A unique user name opted for by the customer.
`email`	Customer's email id to be used later in the authentication phase.
`password`	The customer's password is used for authentication later.

- **User authentication:** To authenticate users, we use the following API, which we assume is the basic authentication mechanism.

```
authenticateUser(username, password)
```

Payment processing

In payment processing, we have the following APIs, which perform different task's including the following:

- **Payment authorization:** This API requests authorization for a payment transaction. It verifies whether the customer has sufficient funds or credit to cover the transaction amount and associates the transaction with the intended merchant. Upon successful verification, it returns an authorization code for the specified payment details.

```
authorizePayment(amount, card_number, expiration_date, CVV, merchant_id)
```

Let's describe the parameters in the following table:

Parameter	Description
amount	The amount to be paid by the customer for a purchase.
card_number	16-digit customer's payment card number.
expiration_date	The expiry date of the payment card.
CVV	A 3-digit card verification value of the payment card.
merchant_id	A unique identifier for the merchant receiving the payment. Used to route the transaction and reserve funds for the correct merchant account.

- **Payment capture:** This API enables us to capture authorized funds after successful payment authorization, complete the transaction, and transfer the funds to the merchant's account.

```
capturePayment(authorization_code, amount)
```

Parameter	Description
authorization_code	The authorization code provided by the `authorizePayment` API upon successful verification.
amount	The amount to be deducted from the customer's account and transferred to the merchant's account.

- **Verify payment status:** This API checks the status of a payment transaction, allowing us to confirm whether a payment was successful or declined.

```
checkPaymentStatus(transaction_id)
```

- **Transaction history:** This API returns a customer's transaction history during the specified date range.

```
getTransactionHistory(customer_id, start_date, end_date)
```

Parameter	Description
customer_id	The customer's ID is the one who has requested transaction history.
start_date	The start date of the transaction is to be displayed to a customer.
end_date	The end date of the transaction to be displayed to a customer.

We have explored the payment system's API design. Now, let's examine the storage schema that supports these APIs in the following section:

2.3 Storage schema

Each of the above APIs requires support from the database. Let's understand how the payment system stores different types of information:

We have explored the payment system's storage schema. Now, let's move to its detailed design in the following section:

Transaction	
transaction_id:	VARCHAR (20)
customer_id:	VARCHAR (20)
merchant_id:	VARCHAR (20)
payment_method:	VARCHAR (50)
amount:	DECIMAL(10,2)
timestamp:	DATETIME
status:	VARCHAR (20)

Customer	
customer_id:	VARCHAR (20)
customer_name:	VARCHAR (50)
description:	VARCHAR (50)
creditcard_on:	VARCHAR (16)

Merchants	
merchant_id:	VARCHAR (20)
business_name:	VARCHAR (50)
business_type:	VARCHAR (50)
description:	VARCHAR (50)

Tokens	
token_id:	VARCHAR (20)
customer_id:	VARCHAR (20)
token_description:	VARCHAR(100)
expiry:	DATETIME

Ledger	
transaction_id:	VARCHAR(20)
transaction_date:	DATE
token_id:	VARCHAR (20)
action:	VARCHAR (50)
description:	VARCHAR (100)

Wallet	
wallet_id:	VARCHAR (20)
creation_date:	DATETIME
update_date:	DATETIME
balance:	DECIMAL (10,2)
merchant_id:	VARCHAR (20)
currency_code:	VARCHAR (3)

Figure 1.5: Payment system storage schema overview

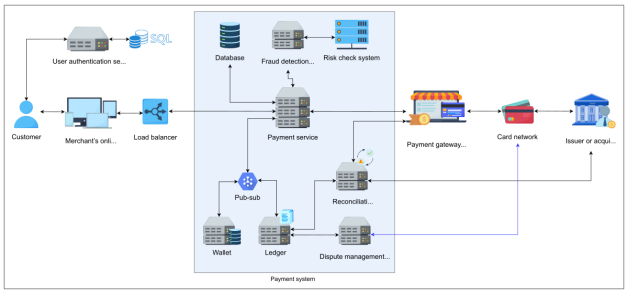

Figure 1.6: Detailed design of the payment system

2.4 Detailed design

Let's examine the detailed design of the payment system. The illustration below highlights the various components involved in the system:

Initially, a customer selects a product or service via the merchant's online store and proceeds to the checkout page, where they provide their payment detail. The payment details include the card number, cardholder name, CVV or CVC(The Card Verification Value (CVV) or Card Verification Code (CVC) is a three- or four-digit security code located on the back of most credit and debit cards.), and expiration date. Before initiating the payment, the customer is authenticated via the user authentication service, which verifies their identity against stored credentials in the SQL database. Once authenticated, and upon clicking the "pay" (or "buy") button, an event is generated that hits the payment service for processing the provided information.

Let's understand the working of each component in the payment System Design to have a clear understanding of the system:

- **Payment service:** The payment service receives and stores the customer-generated payment event in the associated databases. It also performs initial security operations to verify that the request contains no malicious actors or is susceptible to fraudulent activity. It then sends the payment details to the fraud detection service.

- **Fraud detection service:** This service identifies and mitigates fraudulent transactions in real time. It analyzes transaction patterns, flags suspicious activities, and works with the risk check system to block high-risk payments while ensuring that legitimate transactions are not mistakenly flagged as fraudulent. Transactions flagged as suspicious may be rejected or require additional authentication.

- **Risk check system:** The risk check system works alongside the fraud detection service to assess each transaction's risk level. It evaluates multiple factors, including device fingerprinting, transaction history, location-based risk analysis, and velocity checks. Based on the analysis, it assigns a risk score to the transaction and determines whether to proceed, challenge, or block the payment.

  Quiz: Question 1: If a fraud detection system is already in place, what additional value does a dedicated risk check system provide? **Answer:** While a fraud detection system focuses primarily on identifying and preventing malicious or unauthorized activities, a risk check system serves a broader purpose by evaluating the overall risk profile of a transaction or user, even when no fraud is detected. This layered approach helps mitigate not only fraud but also financial, operational, and reputational risks, offering a more comprehensive defense strategy.

- **Payment gateway (payment service provider):** The payment gateway performs extensive security checks on the request. Due to its complexity, a third party often performs the risk check, a specialized process. The payment gateway is primarily responsible for moving money from the customer's account to the merchant's. Since the payment gateway also acts as a PSP, it offers some secondary services, such as maintaining sessions during the payment process, handling different requests, such as refunds, generating invoices, etc.

- **Card networks:** Since customers normally pay via their credit or debit cards, there should be a mechanism to verify card information. This verification is carried out via the card network. The interaction between a payment gateway and the card network takes place via the APIs provided by the card network.

- **Wallet:** When the payment gateway successfully processes the payment, the payment service (acting as the coordinator) updates the merchant's wallet in the database to reflect the new balance. This wallet is used to track the merchant's total revenue. Sometimes, multiple orders from different sellers might occur in a single purchase. The payment service processes each such order separately. Each seller's wallet is updated after receiving the buyer's account payment.

- **Ledger:** When the payment service updates the wallet, the ledger service appends a new immutable entry to the ledger database. This entry typically includes the transaction ID, amount, currency,

involved parties, transaction type (payment or refund), timestamp, and status. These records provide a reliable and auditable trail of all financial operations.

Quiz: Question 1: Where are the customer's payment details encrypted during a purchase? **Answer:** Customer's payment details in the system are encrypted at several key points: 1. Client-side encryption: Payment details are encrypted on the customer's device (e.g., computer or smartphone)

before being transmitted over the internet. 2. During data transmission: Encrypted connections (SSL/TLS) secure the transmission of payment details between the customer and the payment system's servers. 3. Payment gateway: The payment gateway also encrypts payment details to facilitate customer and merchant transactions. 4.

Storage encryption: Payment details may be temporarily stored and should be encrypted at rest for security. For example, when the payment service receives a payment event, it stores the event details in the database after applying the necessary encryption. Encryption is a vital security measure, complemented by access controls and compliance with industry standards like #key# PCI DSS: PCI DSS stands for Payment Card Industry Data Security Standard. It is a set of security standards and requirements designed to ensure the secure handling of credit and debit card information during payment transactions. #key#, to safeguard customer payment information.

- **Reconciliation system:** Reconciliation in the payment system is the process of matching and verifying financial records to ensure that all transactions are accurately accounted for and that there are no discrepancies between different records and statements. It is a critical financial management component that allows the ledgers to align with the payment gateway (or PSP) transaction records. Reconciliation is particularly important in payment systems because it helps identify and resolve errors, fraud, and discrepancies.
  Periodically, the PSPs send the settlement file to the clients at the day's end. The settlement file includes the balance of the account and all the transactions that occurred on that account during the day. The reconciliation system analyzes the settlement file and reconciles the information in the settlement file with the information in the ledger system. It also helps us to verify that the payment system is consistent internally.
  For example, if a merchant received three payments during the day—one for $50, another for $100, and a third for $75—the settlement file from the PSP would list these transactions and a total settlement amount of $225. The reconciliation system cross-checks this file against the internal ledger entries for the same merchant. The data is reconciled if all three transactions exist in the ledger with matching amounts and timestamps. If there's a mismatch, the system flags it for further investigation.

  Quiz: Question 1: What happens if a mismatch is found during reconciliation? **Answer:** If a mismatch is found, the reconciliation system alerts and logs the discrepancy for further investigation, the issue could be missing transactions, duplicate entries, timing delays, or data corruption. The system may: - Temporarily hold settlements until the mismatch is resolved. - Trigger a reprocessing of affected transactions. - Notify the operations team or initiate automated correction workflows,

  depending on the severity.

- **Dispute management service:** This service handles transaction disputes between customers, merchants, and financial institutions. It collects transaction details, collaborates with the issuer bank and card network, and processes chargebacks if necessary. It upholds compliance with financial regulations and maintains dispute records to prevent fraudulent claims.

We have discussed the detailed design, but a few questions could arise while designing a payment system. For example:

1. How should we guarantee transaction completion?

2. How can we handle transient failures, and what strategies should we adopt?
3. How can we avoid persistent failures?

Let's address the above questions one by one:

2.5 Transaction completion

Within the payment system, multiple interconnected services collaborate to execute a transaction. The primary concern is ensuring that requests aren't lost over the network. Another possibility is that messages reach the backend system but encounter unavailable services.

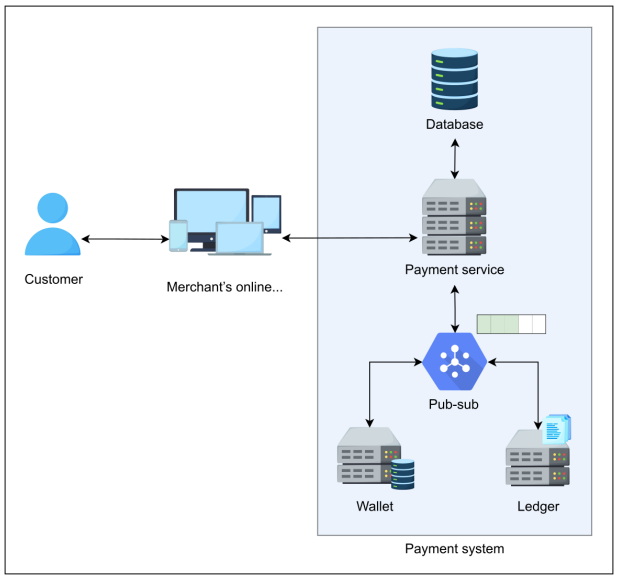

Figure 1.7: Flow of services involved in transaction completion

We employ a pub-sub system like Apache Kafka to secure the completion of transactions. For each order placement or payment, we generate an event within Kafka. This approach aids in managing communication messages to prevent them from being lost in unexpected situations. While it's worth noting that Kafka itself may encounter issues, its current role is relatively straightforward, making its priority notably higher than other business-critical services. The services associated with Kafka consume these messages. A message is marked as consumed only when successfully processed, and its outcome is stored in the wallet's database and ledger.

2.6 Transient failures

A **transient failure** refers to a temporary or short-lived problem or error that occurs in a system or process but does not indicate a permanent issue. Transient failures are typically brief and can be caused by various factors, including network glitches, temporary resource constraints, software bugs, or external factors beyond the system's control. In the payment system, it is important to avoid such temporary issues, and there needs to be a proper mechanism for resolution. To address transient failures, systems often rely on strategies such as:

- Retry strategy
- Timeout
- Fallback mechanism

Let's examine how each of these strategies helps improve resilience in the payment system.

Retry strategy

The retry strategy helps us avoid temporary network problems and ensure that the payment transactions are processed reliably. For this, we implement exponential backoff(If a payment transaction initially fails due to a temporary network problem, the system waits for a short period before attempting to retry. If the retry also fails, the system waits longer before the next attempt.) and set retry limits. The retry limits set a maximum limit on the number of retry attempts for a transaction. After reaching this limit, the request will be marked as failed and can be retried after a specific time interval.

Timeout

The purpose of implementing a timeout is to prevent waiting indefinitely for a response. When the response time becomes excessively long, the operation is terminated, and the request is marked unsuccessful. This mechanism is employed in almost every application to prevent requests from becoming stuck indefinitely. Nonetheless , handling timeouts can be a complex task. Consider a scenario in an online store where a payment request fails for an order. The customer may observe that the purchase order wasn't processed, but various outcomes could occur in the backend, as given below:

1. It's possible that the payment request was effectively initiated, but the response from the payment service might have been lost in transit from the backend.

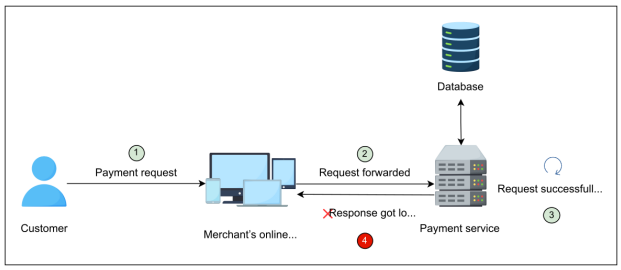

Figure 1.8: Response from the payment service got lost in transit

2. The request is being processed within the payment system and has reached a point where a request timeout has occurred.

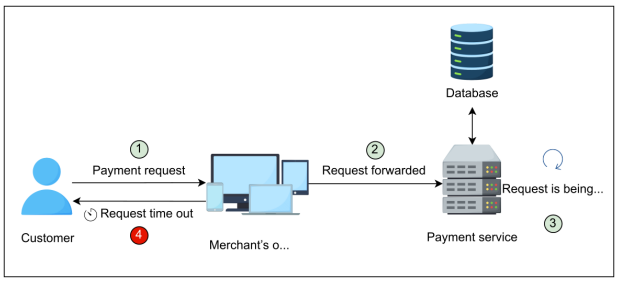

Figure 1.9: Request time out

3. The third scenario could be that the request is lost during transit and has not reached the payment service.

In all the above three scenarios, what status should we give a customer? For example, if we mark the request as failed, the customer might think the order didn't succeed, but it may have succeeded, and they

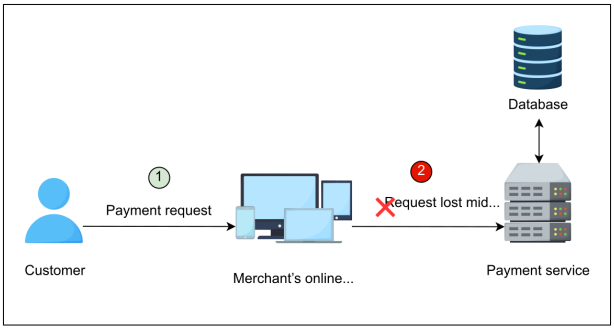

Figure 1.10: Request got lost in midway

could get charged. However, the request was marked as failed due to a lack of response. What happens if the customer retries the operation? This is where **idempotency** comes into play. Idempotency ensures that even if a customer retries a request, the system will handle it in a way that prevents duplicate charges or unintended consequences.

> **Quiz: Question 1:** How to ensure that there are no duplicate payments when a customer hits the buy button many times for a single purchase? **Answer:** We can create a distinct transaction identifier (like an order number or payment ID) for every transaction or purchase, which serves as our idempotence key. This key is stored in our database when the customer initiates the payment and the request's status (for example, succeeded,
>
> failed, or in progress). If a customer accidentally clicks the "buy" button twice or generates a duplicate request for the same purchase, they won't be charged, as the idempotence key in the database prevents multiple charges. **Question 2:** How big should be the time-out value? **Answer:** Deciding the timeout value depends on how complex the payments are and what kind of system we use. For basic online shopping, a small value is usually fine. But we might need a larger value if we are doing something more complex, like sending money internationally or booking a flight.
>
> It's also a good idea to follow the guidelines of the payment gateways we are using. We must balance making it easy for customers and keeping things secure. If we set the value very small, customers might get frustrated. If it's too large, it could be a security risk. As a rule of thumb, think about how long it takes for most customers to finish their payment and use that as a guide.

Fallback

Fallback typically refers to a backup method or alternative process that can be used when the primary payment method or technology fails or encounters difficulties. It enables a service to continue executing even when requests to a dependent service encounter failures. As an example, during the payment process, a request might be dispatched to the fraud check service, which, in turn, may respond with an internal server error. Instead of halting the entire computation due to the absence of a response, we can substitute a fallback value(A fallback value is a predefined or default value that is used when a specific condition or scenario occurs, typically when the system encounters an unexpected situation or cannot retrieve the desired data or perform a certain operation.). For example, in the case of very small transactions, we can permit the transaction to proceed. This approach represents a balance between assuming certain risks and ensuring customer satisfaction by minimizing wait times.

2.7 Achieving requirements

Let's see how the proposed payment System Design satisfies both functional and non-functional requirements.

Achieving functional requirements

Let's examine how the payment system fulfills the functional requirements that we outlined in the previous lesson, in the following table:

Functional Requirements	Techniques
User registration and authentication	• Handled by the merchant's online store. It provides secure login, session management, and user identity validation • Authenticated requests forwarded to the payment service
Payment processing	• Payment processing is handled by the payment service in conjunction with other coordinated services, such as PSP, fraud detection services, and reconciliation services, among others.
Transaction history	• The ledger stores immutable transaction logs, including details such as ID, amount, currency, involved parties, status, and timestamp.
Balance management	• The wallet maintains up-to-date merchant balances by reflecting successful payments and refunds. • The payment service updates the wallet after each transaction. • Ledger provides an auditable balance history for verification
Mobile accessibility	• Enabled by a responsive Merchant's online store UI.

Achieving non-functional requirements

Beyond core features, a payment system must be secure, scalable, and resilient. The table below shows how our proposed design meets these essential non-functional requirements:

2.8 Conclusion

Designing a secure and resilient payment system requires a balance between efficiency, reliability, and fraud prevention. In this chapter, we've explored the entire payment process, from transaction authorization to settlement, while addressing the functional and non-functional requirements that shape such a system. Our proposed design focuses on enabling payments through credit and debit cards. While the desigOur proposed design focuses on enabling payments through credit and debit cards. While the design addresses key components, it's essential to note that certain entities, such as banks, online merchant stores, and risk-check services, are part of larger systems that fall outside the scope of this specific design problem.
A well architected payment system ensures seamless, secure transactions while minimizing risks and maintaining financial integrity. By thoughtfully selecting system components and architectural patterns, businesses can build scalable, robust, and trustworthy payment infrastructures that meet current industry demands and are adaptable to future challenges.

Non-functional Requirements	**Techniques**
Data integrity and security	• Adhere to industry security standards such as PCI DSS (Payment Card Industry Data Security Standard). • Use robust encryption and secure storage mechanisms.
Reliability and availability	• Used redundant services (payment, reconciliation, risk check) to prevent failures. • Replicate data across multiple data centers for high availability. • Utilize local load balancers to exclude failed servers. • Deploy global load balancers to redirect traffic during regional failures.
Scalability	• Scale databases efficiently and cache frequently accessed data to optimize performance. • Implement load balancing to prevent bottlenecks. • Enable monitoring and automated scaling for dynamic resource allocation.
Performance	• Process transactions asynchronously to decouple services. • Ensure sufficient bandwidth during peak times. • Implement transient failure strategies (retry, timeout, and fallback) to ensure timely responses.